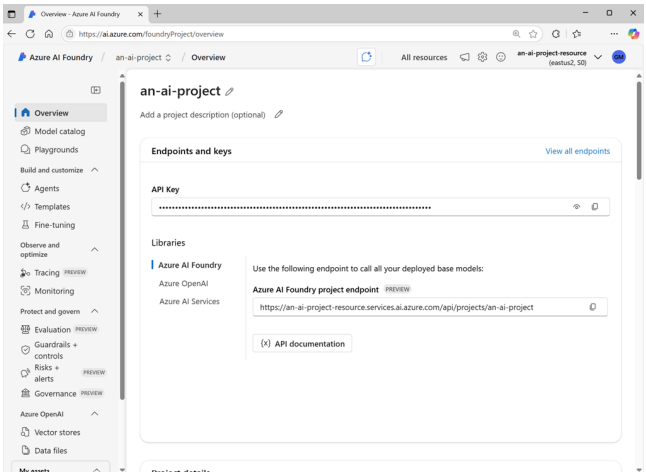
Develop an AI app with the Microsoft Foundry SDK

Wednesday, March 11, 2026 7:28 PM

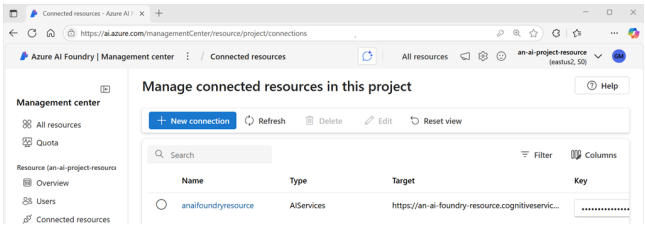
1. Introduction
   a. Developers creating AI solutions with Microsoft Foundry need to work with a combination of services and software frameworks.
   b. The Microsoft Foundry SDK is designed to bring together common services and code libraries in an AI project through a central programmatic access point, making it easier for developers to write the code needed to build effective AI apps on Azure.

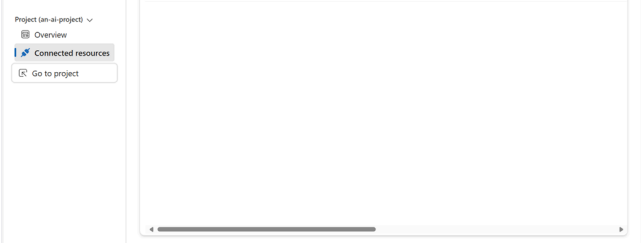
2. What is the Microsoft Foundry SDK
   a. Microsoft Foundry provides a REST API that you can use to work with AI Foundry projects and the resources they contain. Additionally, multiple language-specific SDKs are available, enabling developers to write code that uses resources in A Microsoft Foundry project in their preferred development language. With A Microsoft Foundry SDK, developers can create applications that connect to a project, access the resource connections and models in that project, and use them to perform AI operations, such as sending prompts to a generative AI model and processing the responses.
   b. The core package for working with projects is the Azure AI Projects library, which enables you to connect to A Microsoft Foundry project and access the resources defined within it. Available language-specific packages the for Azure AI Projects library include:
      a. https://pypi.org/project/azure-ai-projects/
   c. Using the SDK to connect to a project:
      a. The first task in most Microsoft Foundry SDK code is to connect to A Microsoft Foundry project. Each project has a unique endpoint, which you can find on the project's Overview page in the Microsoft Foundry portal.
      b.
      c. 

      d. The project provides multiple endpoints and keys, including:
         i. An endpoint for the project itself; which can be used to access project connections, agents, and models in the Microsoft Foundry resource.
         ii. An endpoint for Azure OpenAI Service APIs in the project's Microsoft Foundry resource.
         iii. An endpoint for Foundry Tools APIs (such as Azure Vision and Azure Language) in the Microsoft Foundry resource.
      e. You can use the project endpoint in your code to create an AIProjectClient object, which provides a programmatic proxy for the project, as shown in this Python example:
      f.
```python
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient
...

project_endpoint = "https://......"
project_client = AIProjectClient(
    credential=DefaultAzureCredential(),
    endpoint=project_endpoint)
```
      g. The code uses the default Azure credentials to authenticate when accessing the project. To enable this authentication, in addition to the azure-ai-projects package, you need to install the azure-identity package: >> pip install azure-identity
      h. To access the project successfully, the code must be run in the context of an authenticated Azure session. For example, you could use the Azure command-line interface (CLI) az-login command to sign in before running the code.

3. Work with project connections
   a. Each Microsoft Foundry project includes connected resources, which are defined both at the parent (Microsoft Foundry resource or hub) level, and at the project level. Each resource is a connection to an external service, such as Azure storage, Azure AI Search, Azure OpenAI, or another Microsoft Foundry resource.



b. 

c. An official UI template for Microsoft: https://azure.github.io/ai-app-templates/

d. With the Microsoft Foundry SDK, you can connect to a project and retrieve connections; which you can then use to consume the connected services.

e. For example, the AIProjectClient object in Python has a connections property, which you can use to access the resource connections in the project. Methods of the connections object include:

   a. connections.list(): Returns a collection of connection objects, each representing a connection in the project. You can filter the results by specifying an optional connection_type parameter with a valid enumeration, such as ConnectionType.AZURE_OPEN_AI

   b. connections.get(connection_name, include_credentials): Returns a connection object for the connection with the name specified. If the include_credentials parameter is True (the default value), the credentials required to connect to the connection are returned - for example, in the form of an API key for a Foundry Tools resource.

c. The connection objects returned by these methods include connection-specific properties, including credentials, which you can use to connect to the associated resource.

The following code example lists all of the resource connections that have been added to a project:

Python Copy

```python
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient

try:

    # Get project client
    project_endpoint = "https://....."
    project_client = AIProjectClient(
            credential=DefaultAzureCredential(),
            endpoint=project_endpoint,
        )

    ## List all connections in the project
    connections = project_client.connections
    print("List all connections:")
    for connection in connections.list():
        print(f"{connection.name} ({connection.type})")

except Exception as ex:
    print(ex)
```

4. Create a chat client

   a. While you can use the Azure OpenAI SDK, to connect "directly" to a model using key-based or Microsoft Entra ID authentication; when your model is deployed in a Microsoft Foundry project, you can also use the Microsoft Foundry SDK to retrieve a project client, from which you can then get an authenticated OpenAI chat client for any models deployed in the project's Microsoft Foundry resource. This approach makes it easy to write code that consumes models deployed in your project, switching between them easily by changing the model deployment name parameter.

   b. The following Python code sample uses the **get_openai_client()** method to get an OpenAI client with which to chat with a model that has been deployed in the project's Microsoft Foundry resource.

Python Copy

```python
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient
from openai import AzureOpenAI

try:

    # connect to the project
    project_endpoint = "https://......"
    project_client = AIProjectClient(
            credential=DefaultAzureCredential(),
            endpoint=project_endpoint,
        )

    # Get a chat client
    chat_client = project_client.get_openai_client(api_version="2024-10-21")

    # Get a chat completion based on a user-provided prompt
    user_prompt = input("Enter a question:")

    response = chat_client.chat.completions.create(
        model=your_model_deployment_name,
        messages=[
            {"role": "system", "content": "You are a helpful AI assistant."},
            {"role": "user", "content": user_prompt}
        ]
    )
    print(response.choices[0].message.content)

except Exception as ex:
    print(ex)
```

5. Exercise - Create a generative chat app

6. Module Assessment

   a. AIProjectClient is used to connect to an Azure AI Foundry project and acts as the main proxy object for interacting with project resources.

   b. When creating an AIProjectClient, you provide the project endpoint URL along with credentials.

c. Chat interactions with deployed models (GPT-4, GPT-4o, etc.) in Azure AI Foundry use the Azure OpenAI SDK.